

Building CNN from Scratch : Hair Type Image Multiclass Classification with Modified Convolutional Neural Network Architecture

Charles Fredric G. Inventado
College of Information and Computing Sciences
University of Santo Tomas
Bulacan, Philippines
charlesfredric.inventado.cics@ust.edu.ph

James Vincent V. Valles
College of Information and Computing Sciences
University of Santo Tomas
Mandaluyong, Philippines
jamesvincent.valles.cics@ust.edu.ph

Abstract— Specific domains value correct hair type prediction using Artificial Intelligence such as e-commerce. Businesses employing a fashion based on applications providing an accurate list of recommendations for a user's personalized hair care is a key to enhancing loyalty and trust to using their brand in the long run. The study includes the changing of different hyperparameters and exploring how these different factors that the researchers manipulated affected the performance of the model by comparing the experiment model against the Baseline CNN Model – involving three Convolution Layers, and then a flattening layer where researchers attempts changing the preprocessing technique, filter size, pooling size, even adding a max pooling to each convolution, learning rate, number of neurons. The researchers also attempted a combination of different experiments such as an increased number of filters along with four convolutional layers. From the results, the final model had the best accuracy with a value of 0.88 while having a validation accuracy of 0.61. The final model also contained a remarkable ROC-AUC value, a training value of 0.97 and a validation value of 0.81, interpreting the confidence of the model to distinguish the different hair types which is important for multiclass classification problems. Suggestions arise to be explored by future researchers to try changing the strides and dilation of the model because these hyperparameters are currently unexplored on the study which may create new opportunities to improve and find a better combination of approaches that could beat the performance of the final model of the researchers.

Keywords— *Convolutional Neural Network, CNN Architecture, CNN Hyperparameter Tuning, Hair Type Classification, Image Preprocessing, Neural Network Design, Feature Extraction, Computer Vision, Pattern Recognition, CNN from Scratch, Baseline CNN Model, Modified Baseline CNN Model, Convolutional Layers, Activation Functions, Flattening Layers, Dense Layers, Neurons, Preprocessing, Training Dataset, Validation Dataset, Performance Metrics, Input Validation*

I. INTRODUCTION

The uprising trend on the use of mobile and website applications for beauty, fashion, cosmetic, and health gives an opportunity to benefits that can be attained on having Artificial intelligence (AI), [1] where subset fields such as Modern Machine Learning (ML) Techniques such as CNN is also included. Automation on the hair type and other functionalities that require AI and ML could improve the

responsiveness of the business to meet the needs of their users and succeed as a customer-centric and data-driven brand. [2] The quick and reliable performance of the models gives a head start as well as an advantage to the businesses who incorporate ML technology against other competitors in the beauty, fashion, cosmetic, and health industries.

The study conducted by these researchers focused on classifying three different hair types (curly, straight, and wavy) while using a Convolutional Neural Network (CNN) that will be implemented first from scratch. Furthermore, the researchers will manipulate different hyperparameters in a CNN and evaluate how using or changing the specific hyperparameters will affect the performance of the CNN model on predicting the hair type.

A. Problem

The researchers were given instructions to build their own CNN model which is to be tuned to identify hair types. There is a need to identify what changes to the CNN model need to be made and what characteristics of the hair the researchers will target in order to get the best results.

B. Dataset

The dataset is provided by the professor, Mr. Jessie James Suarez, as part of summative lab exercise # 3 during the finals period on the course 'Machine Learning'.

The dataset contains 3 different folders labeled 'Curly_Hair', 'Straight_Hair', and 'Wavy_Hair'. These captured photos found on each distinct folder shows images of individuals that contain a common characteristic of what hair type can be described from their hair.

The total number of images after extraction is 994, which composed of :

TABLE 1
List, Quantity, and Definition of Hair Types from the Hair Type Dataset

Classification (Label from Extraction)	Quantity	Definition of the Classification from the Images
Curvy_Hair	339	The individual has curvy hair.
Straight_Hair	324	The individual has straight hair.
Wavy_Hair	331	The individual has wavy hair.

These images will undergo data preprocessing to prepare the data for training and testing of the built CNN model. Primarily used data preprocessing steps such as image resizing, normalization, label encoding, and train-test split are considered to be conducted for the study .

C. CNN Hyperparameters

In CNN, hyperparameters refer to the variables that can be configured before the training procedure of a CNN model starts. These manipulations can influence how the model will learn and perform, thus the possibility of changes on the prediction rates of the hair types as a result.

As a recap, a simple CNN model has an input layer, hidden layer, and output layer. These layers have input neurons, hidden neurons, and output neurons respectively. In every neuron there is an activation function while for its tuning it has a loss function, gradient descent function, and a training rate.

These are the following hyperparameters considered as options to be experimented for the study :

TABLE 2
List of Hyper Parameters Options for Experimentation

Hyperparameter	Definition of the Term (DoT)
Learning Rate	How much the model's parameters (such as weights) are adjusted during each update in the training process.
Batch Size	Number of images that will pass through the CNN model before updating the weights.
Epochs	The number of epochs corresponds to the number of passes through the entire dataset through the model.
Number of kernels	The number of different applied filters to the input to modify or enhance an image. The number of kernels will also correspond to the number of feature maps output.
Size of kernels	Typically a $n \times n$ sized dimension that will be used

	to scan the images on each slide. Where n = number of pixels.
Stride	How much does the kernel will slide across an image and produce an output value at each slide occurrence.
Padding	Pixels added to an image, value that can vary from 0 to 255, typically used to preserve the spatial size of the feature map compared to the input size (considering the stride and kernel size)
Number of convolutional Layers	Number of convolutional operations stacked on the CNN model responsible for the extraction of features on the images.
Pooling Size	Dimensions used for the window during pooling. Also determines how much the feature map or resolution will be reduced.
Pooling Type	Operation applied to each window during pooling. Ex : max, average, and sum pooling.

D. Objectives

In order to identify if our study is a success we've listed down the objectives to aim for when conducting the experiment and its outcomes.

- Apply appropriate data preprocessing techniques to potentially improve the learning outcomes of the CNN model
- Build a Machine Learning Pipeline from the baseline model for the Final CNN model
- Empirically identify the major hyper parameters to tune in the baseline Model to detect the type of hair better.
- Evaluate the effects of the chosen parameters tuned in the CNN Model and describe how it contributed to the performance of the model.
- Obtain at least 50% accuracy and precision when identifying the 3 hair types.

E. Significance of the Study

By conducting the experiment the researchers get better at tuning CNN models to detect what characteristics the object has, and better identify on what type of hair the person has, and detail the methodology the researchers did so that future researchers would know how to tune models to detect the characteristic of a different subject.

II. METHODOLOGY

The Methodology section discusses how the neural network will be built from scratch, exploring different components that can be manipulated and implemented on a CNN architecture. The section also includes the different approaches made by the researchers to empirically evaluate the different hyperparameters in preparation of the experimentation of the neural network.

These include the data preprocessing, creation of the CNN model, training of the model, and detailed performance metrics on the evaluation of the neural network.

A. Data Preprocessing

Omitting the need for Data Augmentation

Given the constraints of the dataset and the initial scope of this research focused on foundational CNN architecture learning, the researchers opted for a minimal preprocessing approach beyond basic resizing and normalization. Data augmentation was intentionally omitted to maintain a direct mapping between the original data characteristics and the learned features, allowing for a clearer understanding of the network's inherent capabilities

TABLE 3
Image Distribution of the Hair Types Before Image Checking

	# of images	Percentage
Curvy_Hair	339	$339 / 994 * 100 = 34.10\%$
Straight_Hair	324	$324 / 994 * 100 = 32.60\%$
Wavy_Hair	331	$331 / 994 * 100 = 33.30\%$
Total	994	100%

From table 3, before image checking, the distribution of the different classes only ranged from 32 to 34 %, which can be described as the data to be balanced. Which supports the researchers decision to omit the need for data augmentation.

Further note on the later parts of the study that there were no issues if checking of image distribution was done after the image validation as only 9 images were affected, less than 0.01 % of the images.

Handling Missing Values or Images

Similarly, the dataset exhibited no significant missing values, rendering specific handling techniques unnecessary. Other preprocessing techniques for image validation such as checking file compatible file types, and other image checking techniques are conducted and elaborated on the experimentation of the study.

Finally, the raw pixel data, once normalized, was deemed sufficient for the initial investigation, with feature

scaling or engineering reserved for potential future explorations aimed at optimizing performance beyond the core architectural study.

Normalization and Rescaling

Normalization and rescaling of pixel values of the images are to be conducted before the feature extraction of the CNN model that will improve the learning performance of the features. Different approaches may be conducted during the experimentation to find out which method of normalizing or scaling to use that can aid in developing a competitive CNN model performance from scratch such that metrics are significantly better when going to be compared soon to the baseline CNN model .

B. CNN Architecture

The initial CNN architecture provided as our baseline model consists of three convolutional layers with progressively smaller kernel sizes (16×16, 8×8, and 4×4) and an increasing number of filters (4, 8, and 16), all using ReLU activation, valid padding, and a stride of 1. After feature extraction, the network applies global average pooling to reduce spatial dimensions before passing the data through a dense layer with 64 neurons and a final softmax output layer with 3 units for classification. The model is trained for 50 epochs using the Adam optimizer with a learning rate of 0.001 and categorical cross-entropy loss. To improve performance, we will experiment with several modifications, including adjusting kernel sizes (testing smaller 3×3 and 5×5 filters for finer feature detection), increasing filter counts (e.g., 32 or 64 filters per layer to capture more complex patterns), and testing different padding strategies (comparing 'valid' vs. 'same' padding to see how border information affects accuracy). Additionally, we will explore adding max-pooling layers to reduce computational load, integrating batch normalization to stabilize training, and trying alternative activation functions like LeakyReLU or Swish to improve gradient flow. Each change will be evaluated based on validation accuracy, training speed, and model complexity to determine the most effective architecture for our task.

C. CNN Model Implementation and Training

The researchers implement the CNN model using TensorFlow/Keras, starting with the baseline architecture described and then incorporating the proposed modifications to enhance performance. The model is trained on the preprocessed dataset with a 80-20 train-validation split, using early stopping to prevent overfitting and ModelCheckpoint to save the best weights. We monitor key metrics like accuracy, loss, and training time, comparing the baseline against modified architectures (e.g., with added pooling, batch normalization, or adjusted kernel sizes) to determine the optimal configuration. Hyperparameters such as learning rate, batch size, and optimizer settings are fine-tuned iteratively, with results logged for analysis. Finally, the best-performing model is evaluated on the test set to assess generalization,

followed by visualizations of training curves and confusion matrices to interpret performance.

D. CNN Evaluation Metrics

The metrics to be used are according to :

Type of problem the researchers are solving – which is an image multi-class classification type of problem, with three categories in total. Determining whether the image of an individual contains curvy, straight, or wavy hair.

As the objectives of the research also mentioned is a classification problem, these are the following metrics that will aid on evaluating the performance of built CNN model :

Confusion Matrix

In the context of the study, the confusion matrix summarizes the actual type of hair compared to the predicted hair type. As the dataset contains 3 categories (curvy, straight, and wavy), the confusion matrix will be 3 x 3 in size. The description of each cell are as follows :

TABLE 4
Confusion Matrix for Multiclass Classification of Three Hair Types

Actual/Predicted	Curvy (Type A)	Straight (Type B)	Wavy (Type C)
Curvy (Type A)	True Positive_A	False Positive_A_B	False Positive_A_C
Straight (Type B)	False Positive_B_A	True Positive_B	False Positive_B_C
Wavy (Type C)	False Positive_C_A	False Positive_C_B	True Positive_C

Explanation of the Confusion Matrix cells :

TP = True Positives, FP = False Positives

* True Positives (TP) are the correctly predicted har types, meanwhile False Positives (FP) are incorrectly predicted hair types wherein the format description is in the form : did not predict Type X correctly (instead incorrectly predicted as Type Y)

TP_A: True positives for Type A (correctly predicted as Type A).

FP_A_B: False positives for Type A (incorrectly predicted as Type B)

FP_A_C: False positives for Type A (incorrectly predicted as Type C)

TP_B: True positives for Type B (correctly predicted as Type B)

FP_B_A: False positives for Type B (incorrectly predicted as Type A)

FP_B_C: False positives for Type B (incorrectly predicted as Type C)

TP_C: True positives for Type C (correctly predicted as Type C)

FP_C_A: False positives for Type C (incorrectly predicted as Type A)

FP_C_B: False positives for Type C (incorrectly predicted as Type B)

Accuracy

Accuracy measures the model's overall effectiveness of the correct classification of the presence and absence of a specific hair type. The corresponding formula for accuracy is :

$$A = \frac{TP+TN}{TP+TN+FP+FN}$$

Where :

A = Accuracy of the specific hair type

TP = True Positive of the specific hair type

TN = True Negatives of the specific hair type

FP = False Positives of the specific hair type

FN = False Negatives of the specific hair type

Precision

Precision determines whether the model does well classifying a specific hair type correctly. Precision refers to the number of correct predictions of the specific hair type over the number of correct and incorrect predictions on the presence of a specific hair type.

$$P = \frac{TP}{TP + FP}$$

P = Precision of the specific hair type

TP = True Positives of the specific hair type

FP = False Positives of the specific hair type

Recall or Sensitivity

Recall refers to how close the number of predictions that the model made to the actual number of cases a specific hair type is present on the dataset. Recall counts the number of correctly identified predictions to the proportion of the number of predictions that a specific hair type is present.

$$R = \frac{TP}{TP + FN}$$

Where :

R = Recall of the specific hair type

TP = True Positives of the specific hair type

FN = False Negatives of the specific hair type

F1-Score

F1-score is usually used for imbalanced data and can be used when precision and recall are equally as important. F1-score is the balance between precision and recall which can be used to balance the number of false positives and false negatives.

$$F1 = \frac{2 \times P \times R}{P + R}$$

Where :

F1 = F1-score of the specific hair type

P = Precision of the specific hair type

R = Recall of the specific hair type

Specificity

Specificity refers to how the model performs on classifying negative instances or when the specific hair type correctly predicts the absence of a specific hair type. The specificity is calculated by the number of correct absent hair type predictions to the proportion of both correctly absent and incorrectly predicted that a case is present.

$$S = \frac{TN}{TN + FP}$$

Where :

S = Specificity of the specific hair type

TN = True Negatives of the specific hair type

FP = False Positives of the specific hair type

ROC-AUC

(Receiver Operating Characteristic - Area Under Curve)

The metric plots the true positive rate against the false positive rate. ROC-AUC is calculated from the ROC curve, which is in a form of graph that illustrates the diagnostic ability of a binary classifier system, showing how it is evaluated at different classification thresholds.

A higher AUC value can be interpreted that a model will distinguish better correctly between a randomly chosen positive case and a randomly chosen negative case.

Note regarding ROC :

As a multi-class problem, the researchers decided to do the One vs Rest approach where ROC value is computed separately for each class, treating a specific type as a positive class and the other two as negative class, resulting in three ROC curves.

III. EXPERIMENTS

The third section includes the execution of different modifications with an aim to understand the impact of different hyperparameters on a CNN model.

Also the current section is expected to have what experiments the researchers decided to do, explain the process, and give the reasoning behind doing those decisions. Starting with the processes conducted from the baseline model first, ranging from the sections A up to H.

A. Image Acquisition

The images were gathered first from the hair_types file provided to serve as the dataset to be used to be fed for the CNN model.

There were two possible ways to store the dataset, first is to compile it through google drive and distinguish the file location of the hair_types folder dataset. Second method is having the dataset locally finding it through a directory within the researchers local system

```
import os

folder_name = 'hair_types'

if os.path.isdir(folder_name):
    print(f"Contents of '{folder_name}':")
    for item in os.listdir(folder_name):
        print(f"- {item}")
else:
    print(f"Folder '{folder_name}' not found in the same directory.")
```

Fig. 1. Code Snippet for using hair_tpyes Dataset from Localhost

Observed from Fig. 1., the researchers selected the second option and used the local directory and used the dataset locally instead of using gdrive. Although there is no significant difference yet between the two options, ending up to the researchers preference on opting to do the second options.

B. Handling Incompatible Images

```
[ ] from pathlib import Path
import imghdr
import os

data_dir = "hair_types" # Directly target the hair_types folder
image_extensions = {".jpg", ".jpeg", ".png", ".gif"}

for hair_type_folder in os.listdir(data_dir):
    hair_type_path = os.path.join(data_dir, hair_type_folder)
    if os.path.isdir(hair_type_path):
        print(f"Processing folder: {hair_type_path}")
        for filename in os.listdir(hair_type_path):
            filepath = os.path.join(hair_type_path, filename)
            if Path(filepath).suffix.lower() in image_extensions:
                print(f"Trying to open: {filepath}")
                try:
                    img_type = imghdr.what(filepath)
                    if img_type is None:
                        print(f"{filepath} is not a recognized image format")
                except FileNotFoundError as e:
                    print(f"Error opening {filepath}: {e}")
```

Fig. 2. Code Snippet for Identifying Compatible and Invalid Images

After having the dataset, the researchers need to ensure that the images contain only the compatible image types. Example again from the demo file, which is the baseline model, have constraints that removed the image on being used for the experiment.

Using the code from Fig. 2, describes the scenarios when the image is incompatible. First, when the image was of .webp type, the code implements in a way that does not accept these types of images. webp images can be deemed reasonable to exclude for being fed to the model because it can create inconsistencies due to webp containing 4 channels with red, green, blue, alpha or transparency meanwhile the common .jpg only have 3 which are red, green, and blue. [3] Second, there are rare instances stating that it says an image is "is not an image" that researchers must be cautious also so it is advised also to be removed from the experiment. With some time researching the essential of having the correct image types and validity, [4] we are confident to proceed after checking we have valid images to be fed on the Tensorflow application.

After the process of handling these incompatible images, the number of total images was reduced from the original 994 down to 985 only, nine images were affected and removed.

C. Data Splitting

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

image_size = (64, 64)
batch_size = 32

train_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "hair_types/",
    validation_split=0.2,
    subset="training",
    seed=1337,
    image_size=image_size,
    batch_size=batch_size,
    labels='inferred',
    label_mode='categorical'
)

val_ds = tf.keras.preprocessing.image_dataset_from_directory(
    "hair_types/",
    validation_split=0.2,
    subset="validation",
    seed=1337,
    image_size=image_size,
    batch_size=batch_size,
    labels='inferred',
    label_mode='categorical'
)
```

Fig. 3. Code Snippet for Data Splitting

The common 80%-20% train-test split was used on the baseline model that can be seen for Fig. 3. As to repeat, there were 985 files found belonging to the 3 classes as mentioned from all curly, straight, and wavy hair combined.

The result of the process was 788 files are going to be used for training of the model. Meanwhile, 197 files are assigned for validation.

D. Image Visualization

```
import matplotlib.pyplot as plt
import numpy as np

plt.figure(figsize=(10, 10))
for images, labels in train_ds.take(1):
    for i in range(9):
        ax = plt.subplot(3, 3, i + 1)
        plt.imshow(images[i].numpy().astype("uint8"))
        plt.title(int(np.argmax(labels[i])))
        plt.axis("off")
plt.show()
```

Fig. 4. Code Snippet for Checking of Data Labelling

The images were proceeded to be visually checked in batches, making sure that the images were correctly formatted before the training process. The checking of correct labels is also of concern whereas an image containing a value of 0 is curly, 1 is straight, and 2 is wavy hair. Refer to Fig. 5. for the sample result.

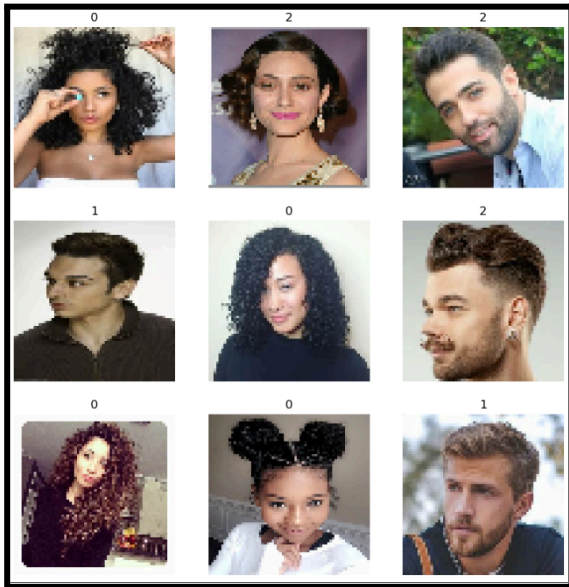


Fig. 5. Results of Checking of the Data Labels

E. Implementing Model Evaluation Metrics

```
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

def evaluate_model(model, dataset, dataset_name="Dataset"):
    y_true = []
    y_pred = []
    y_prob = []

    for images, labels in dataset:
        preds = model.predict(images, verbose=0)
        y_prob.extend(preds)
        y_pred.extend(np.argmax(preds, axis=1))
        y_true.extend(np.argmax(labels.numpy(), axis=1))

    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    y_prob = np.array(y_prob)

    print("\n==== {dataset_name} Evaluation =====")
    print("Classification Report:")
    print(classification_report(y_true, y_pred))

    # Confusion matrix for specificity
    cm = confusion_matrix(y_true, y_pred)
    FP = cm.sum(axis=0) - np.diag(cm)
    FN = cm.sum(axis=1) - np.diag(cm)
    TP = np.diag(cm)
    TN = cm.sum() - (FP + FN + TP)
    specificity = np.where((TN + FP) != 0, TN / (TN + FP), 0.0)
    print("Specificity (per class):", specificity)

    # ROC AUC (macro average for multiclass)
    roc_auc = roc_auc_score(y_true, y_prob, multi_class='ovr', average='macro')
    print("ROC-AUC (macro average):", roc_auc)

    return {
        "classification_report": classification_report(y_true, y_pred, output_dict=True),
        "specificity": specificity,
        "roc_auc": roc_auc
    }
```

Fig. 6. Code Snippet for Implementing Evaluation Metrics

For the evaluation metrics are also to be implemented, here is a better visualization on Fig. 6 showing similar to a summary table of the different metrics that is organized to be pleasantly viewed by the readers of the conducted study.

```
results_baseline_train = evaluate_model(model, train_ds, dataset_name="Baseline Model - Training")

==== Baseline Model - Training Evaluation ====
Classification Report:
      precision    recall  f1-score   support

     0       0.69       0.79       0.74       276
     1       0.59       0.54       0.56       252
     2       0.58       0.54       0.56       260

 accuracy         0.62         0.62         0.63       788
  macro avg       0.62         0.62         0.62       788
 weighted avg     0.62         0.63         0.62       788

Specificity (per class): [0.61854688 0.62276119 0.60492424]
ROC-AUC (macro average): 0.7960347874058343
```

Fig. 7. Result Visualization of Evaluation Metrics

From Fig. 7, includes commonly used evaluation CNN metrics such as precision, recall, f1-score, and support. Support refers to the number of actual occurrences or true samples fed to the model. Macro average and weighted average also is considered. Lastly, accuracy of the model in general, specificity too, and the ROC-AUC.

These metrics are included in the built-in package, but there are instances where we may disregard the macro average and weighted average results, although not initially thought of by the researchers, were considered to be analyzed on some experiments of the study.

F. Baseline Model Architecture Implementation

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from keras.models import Sequential
from keras.layers import Dense
from keras.utils import model_to_dot
from IPython.display import SVG

model = Sequential()
model.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model.add(layers.Rescaling(1.0 / 255))

model.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model.add(layers.Activation("relu"))

model.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model.add(layers.Activation("relu"))

model.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model.add(layers.Activation("relu"))

model.add(layers.GlobalAveragePooling2D())
model.add(layers.Dense(64))
model.add(layers.Activation("relu"))
model.add(layers.Dense(3))
model.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model, to_file='model_test.png', show_shapes=True)

epochs = 50

model.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 8. Code Snippet of the Baseline CNN Model

From Fig. 8, the baseline model accepts 3 channels of 64 by 64 pixels of RGB images. With three convolutional layers, first containing 4 filters, 16 x 16 sized kernel, and containing ReLU (Rectified Linear Unit) activation layer. Second convolutional layer contains more filters with 8, reduced size of 8x8 kernel, and also ReLU activation function. The last convolution layers contains most filters with 16, most small kernel size with 4x4 and ReLU activation function as well.

There is also a global average pooling layer that reduces the spatial dimensions, where the feature maps are converted to a single vector which is the flattening layer stage. The fully connected layers also contain ReLU with dense(64) extracting the deep feature relationships and lastly the dense(3) with SoftMax responsible for the multi-classification of different hair types output.

The number of epochs is 50, which means that the training on the dataset will occur 50 times. The learning rate is “1e-3” or approximately 0.001 with Adam optimizer to ensure learning is adaptive. The loss function is binary cross entropy loss, where 0 means a correct prediction and 1 refers to an incorrect prediction, meaning there is additional loss if an incorrect production occurred on the multiclass classification model. Metrics such as accuracy of the model are produced too.

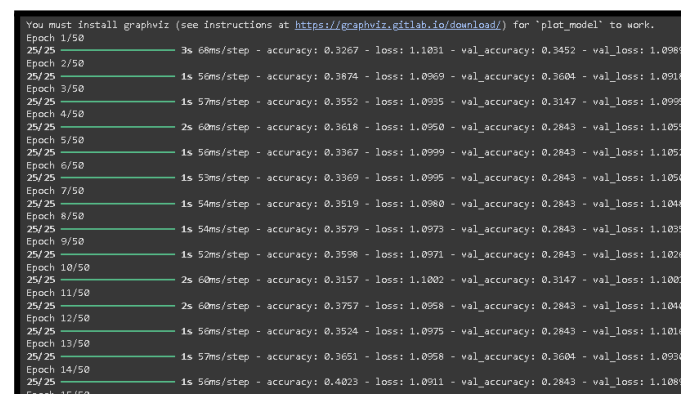


Fig. 9. Result Visualization of Evaluation Metrics

From Fig. 9, is an example of how the model performed differently through different epochs starting from 1 until the number of cycles reaches 50. The time of the training around 50 to 75 ms/step is considered to be good (at least on the researchers perspective), as personally the other research member experienced higher training time of reaching 300 ms and above. (researchers are not sure of the root reason of the problem of the slower training time, thankfully still the one researcher is able to conduct the study with quicker time competitively) accuracy and loss function of both training and validation datasets were gathered for each epoch on the study.

G. Comparing Training and Validation Performance

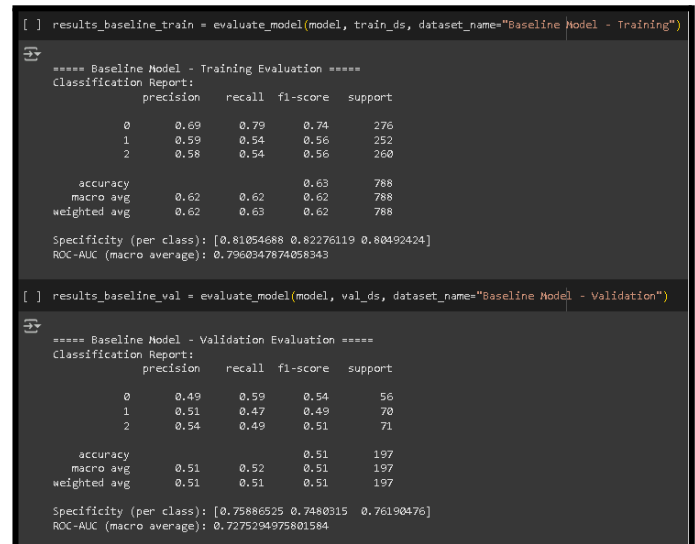


Fig. 10. Training vs. Validation Performance of the Baseline Model

The Fig. 10, shows the performance of the training model against the validation model that is used for comparison whether the current model can generalize well on new, unseen data.

H. Comments on the Model Performance

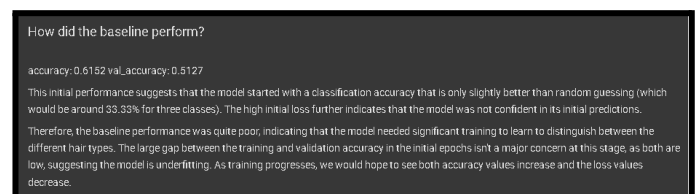


Fig. 11. Training vs. Validation Performance of the Baseline Model

After an experiment is done for the baseline mode, comments are also included describing how well the model performed and additionally the expectations and outcomes the researchers aim to achieve on the following experiments.

I. Experiment 1.1 / Other Preprocessing Methods for Images 1 / Different Image Rescaling

Section I starts the experimentation of different hyperparameters and determines the effect on the model with changed hyperparameters performed against the baseline model that does not have any modifications.

```

model_exp1_1 = Sequential()
model_exp1_1.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp1_1.add(layers.Normalization(mean=0.0, variance=1.0)) # Replaced the existing Rescaling layer with this

model_exp1_1.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp1_1.add(layers.Activation("relu"))

model_exp1_1.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp1_1.add(layers.Activation("relu"))

model_exp1_1.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp1_1.add(layers.Activation("relu"))

model_exp1_1.add(layers.GlobalAveragePooling2D())
model_exp1_1.add(layers.Dense(64))
model_exp1_1.add(layers.Activation("relu"))
model_exp1_1.add(layers.Dense(3))
model_exp1_1.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp1_1, to_file='model_test_1_1.png', show_shapes=True)

epochs = 50

model_exp1_1.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp1_1.fit(train_ds, epochs=epochs, validation_data=val_ds)

```

Fig. 12. Code Snippet for Rescaling Layer Modification with Normalization

Modifying the baseline model on Fig. 12, the first modification experiment on the hyperparameter is the changing of the rescaling layer, where from layers.Rescaling(1.0/255) was changed to layers.Normalization(mean=0.0, variance=1.0). On the specific changes, the researchers aim to find out how rescaling preprocessing technique can differ from using normalization preprocessing technique.

J. Experiment 1.2 / Other Preprocessing Methods for Images 2 / Scaling to the Range [-1, 1]

```

model_exp1_2 = Sequential()
model_exp1_2.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp1_2.add(layers.Rescaling(scale=1.0/127.5, offset=-1)) # Replaced the existing Rescaling layer with this

model_exp1_2.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp1_2.add(layers.Activation("relu"))

model_exp1_2.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp1_2.add(layers.Activation("relu"))

model_exp1_2.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp1_2.add(layers.Activation("relu"))

model_exp1_2.add(layers.GlobalAveragePooling2D())
model_exp1_2.add(layers.Dense(64))
model_exp1_2.add(layers.Activation("relu"))
model_exp1_2.add(layers.Dense(3))
model_exp1_2.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp1_2, to_file='model_test_1_2.png', show_shapes=True)

epochs = 50

model_exp1_2.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp1_2.fit(train_ds, epochs=epochs, validation_data=val_ds)

```

Fig. 13. Code Snippet for Rescaling Layer Modification with Scaling Range to [-1, 1]

Modified again observed on Fig. 13, the rescaling layer but now the rescaling has changed from "layers.Rescaling(1.0/255)" to "layers.Rescaling(1.0/127.5, offset=-1)". The further explanation of the changes is that the range of the rescaling changed from [0, 1] to [-1, 1]. The researchers aim to see how the non-centered inputs such as [0, 1] can perform differently from the zero-centered inputs [-1, 1] on the model performance.

K. Experiment 1.3 / Other Preprocessing Methods for Images 3 / Using Integer Lookup for Faster Rescaling

```

# Define a lookup table to scale integers from 0-255 to 0-1
scale_lookup = tf.constant([i / 255.0 for i in range(256)], dtype=tf.float32)

def scale_image(image):
    # Use tf.gather to perform the lookup on each pixel value
    scaled_image = tf.gather(scale_lookup, tf.cast(image, tf.int32))
    return scaled_image

def preprocess(image, label):
    scaled_image = scale_image(image)
    return scaled_image, label

train_ds_scaled_lookup = train_ds.map(preprocess)
val_ds_scaled_lookup = val_ds.map(preprocess)

```

Fig. 14. Code Snippet for Lookup Table

```

model_exp1_3 = Sequential()
model_exp1_3.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
# No explicit Rescaling layer needed now as scaling is in the map function

model_exp1_3.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp1_3.add(layers.Activation("relu"))

model_exp1_3.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp1_3.add(layers.Activation("relu"))

model_exp1_3.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp1_3.add(layers.Activation("relu"))

model_exp1_3.add(layers.GlobalAveragePooling2D())
model_exp1_3.add(layers.Dense(64))
model_exp1_3.add(layers.Activation("relu"))
model_exp1_3.add(layers.Dense(3))
model_exp1_3.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp1_3, to_file='model_test_1_3.png', show_shapes=True)

epochs = 50

model_exp1_3.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp1_3.fit(train_ds_scaled_lookup, epochs=epochs, validation_data=val_ds_scaled_lookup)

```

Fig. 15. Code Snippet for Using Integer Lookup for Faster Rescaling

On Fig. 14 and Fig. 15, the experiment conducted to compare having an integer lookup table for faster rescaling, a map function, can improve the training speed and performance metrics between the two different approaches where baseline is the one with explicitly mentioned rescaling layer.

L. Experiment 2 / Adding Max Pooling

```
model_exp2 = Sequential()
model_exp2.add(keras.Input(shape=image_size + (3,)))
model_exp2.add(layers.Rescaling(1.0 / 255))

model_exp2.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp2.add(layers.Activation("relu"))
model_exp2.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp2.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp2.add(layers.Activation("relu"))
model_exp2.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp2.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp2.add(layers.Activation("relu"))
model_exp2.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp2.add(layers.GlobalAveragePooling2D())
model_exp2.add(layers.Dense(64))
model_exp2.add(layers.Activation("relu"))
model_exp2.add(layers.Dense(3))
model_exp2.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp2, to_file='model_test_2.png', show_shapes=True)

epochs = 50

model_exp2.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp2.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 16. Code Snippet for Adding Max Pooling to each Convolutional Layer

On Fig. 16, max pooling function is added to each convolutional layer, a total of three max poolings. The researchers want to observe how reducing of parameters or decrease of the resolution with the current approach affects the performance of the model. The previous methods on the first experimentation part were all on parts of pre-processing, to make the second experiment different, now the researchers are manipulating the hyperparameters present on the baseline CNN architecture.

M. Experiment 3 / Changing Number of Filters

```
model_exp3 = Sequential()
model_exp3.add(keras.Input(shape=image_size + (3,)))
model_exp3.add(layers.Rescaling(1.0 / 255))

model_exp3.add(layers.Conv2D(filters=8, kernel_size=16, strides=1, padding='valid', dilation_rate=1)) # Increased 4 => 8
model_exp3.add(layers.Activation("relu"))

model_exp3.add(layers.Conv2D(filters=16, kernel_size=8, strides=1, padding='valid', dilation_rate=1)) # Increased 8 => 16
model_exp3.add(layers.Activation("relu"))

model_exp3.add(layers.Conv2D(filters=32, kernel_size=4, strides=1, padding='valid', dilation_rate=1)) # Increased 16 => 32
model_exp3.add(layers.Activation("relu"))

model_exp3.add(layers.GlobalAveragePooling2D())
model_exp3.add(layers.Dense(64))
model_exp3.add(layers.Activation("relu"))
model_exp3.add(layers.Dense(3))
model_exp3.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp3, to_file='model_test_3.png', show_shapes=True)

epochs = 50

model_exp3.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp3.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 17. Code Snippet for Increasing the Number of Filters

On Fig. 17, the number of filters were modified where the number of filters for each convolutional layer were doubled, such that the first convolution layer now contains 8 filters instead of 4, second layer 16 filters instead of 8, and the third layer now have 32 filters instead of 16. In contrast to experiment two on reducing the features, how about experiment three we have a different approach of increasing the feature extraction power of the model. How will the

different metrics differ on the third experiment as researchers ponder about guessing the outcome of the attempt.

N. Experiment 4 / Changing Kernel Size

```
model_exp4 = Sequential()
model_exp4.add(keras.Input(shape=image_size + (3,)))
model_exp4.add(layers.Rescaling(1.0 / 255))

model_exp4.add(layers.Conv2D(filters=4, kernel_size=(3, 3), strides=1, padding='valid', dilation_rate=1)) # Smaller kernel
model_exp4.add(layers.Activation("relu"))

model_exp4.add(layers.Conv2D(filters=8, kernel_size=(3, 3), strides=1, padding='valid', dilation_rate=1)) # Smaller kernel
model_exp4.add(layers.Activation("relu"))

model_exp4.add(layers.Conv2D(filters=16, kernel_size=(3, 3), strides=1, padding='valid', dilation_rate=1)) # Smaller kernel
model_exp4.add(layers.Activation("relu"))

model_exp4.add(layers.GlobalAveragePooling2D())
model_exp4.add(layers.Dense(64))
model_exp4.add(layers.Activation("relu"))
model_exp4.add(layers.Dense(3))
model_exp4.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp4, to_file='model_test_4.png', show_shapes=True)

epochs = 50

model_exp4.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp4.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 18. Code Snippet for Changing to Smaller Kernel Size

Another attempt for hyperparameter changes that is connected to the feature extraction on the model on Fig. 18. After the fourth experiment attempt, the researchers aim to see the impact without having Max pooling layer to each convolutional layer then applying the smaller kernel size.

O. Experiment 5 / Changing Learning Rate

```
model_exp5 = Sequential()
model_exp5.add(keras.Input(shape=image_size + (3,)))
model_exp5.add(layers.Rescaling(1.0 / 255))

model_exp5.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp5.add(layers.Activation("relu"))

model_exp5.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp5.add(layers.Activation("relu"))

model_exp5.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp5.add(layers.Activation("relu"))

model_exp5.add(layers.GlobalAveragePooling2D())
model_exp5.add(layers.Dense(64))
model_exp5.add(layers.Activation("relu"))
model_exp5.add(layers.Dense(3))
model_exp5.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp5, to_file='model_test_5.png', show_shapes=True)

epochs = 50

model_exp5.compile(
    optimizer=keras.optimizers.Adam(1e-4), # Changed learning rate
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

model_exp5.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 19. Code Snippet for Changing to a Smaller Learning Rate

After changing the hyperparameters related to CNN architecture, preprocessing methods, now let us change the learning rate of the CNN baseline model. On Fig. 19, 0.001 now has a learning rate of 0.0001 or 1e-4. The researchers continue to explore the other hyperparameters as it is fun and fascinating learning further about the model and gives the researchers knowledge especially their research is related also to Convolutional Neural Networks and the current task serves as a practice grounds for them which is significantly beneficial to the success of their Thesis.

P. Experiment 6 / Adding More Neurons

```
model_exp6 = Sequential()
model_exp6.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp6.add(layers.Rescaling(1.0 / 255))

model_exp6.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp6.add(layers.Activation("relu"))

model_exp6.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp6.add(layers.Activation("relu"))

model_exp6.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp6.add(layers.Activation("relu"))

model_exp6.add(layers.GlobalAveragePooling2D())
model_exp6.add(layers.Dense(128))
model_exp6.add(layers.Activation("relu"))
model_exp6.add(layers.Dense(3))
model_exp6.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp6, to_file='model_test_6.png', show_shapes=True)

epochs = 50

model_exp6.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp6.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 20. Code Snippet for Adding More Neurons

From Fig. 20, Changing the dense layer size from 64 to 128, in the sixth experiment the researchers wanted to find out whether adding neurons contributes a positive effect on the baseline CNN model.

Q. Experiment 7.1 / Adding an Additional Block 1 / Only Adding an Additional Block

```
model_exp7_1 = Sequential()
model_exp7_1.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp7_1.add(layers.Rescaling(1.0 / 255))

model_exp7_1.add(layers.Conv2D(filters=4, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_1.add(layers.Activation("relu"))

model_exp7_1.add(layers.Conv2D(filters=8, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_1.add(layers.Activation("relu"))

model_exp7_1.add(layers.Conv2D(filters=16, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_1.add(layers.Activation("relu"))

model_exp7_1.add(layers.Conv2D(filters=32, kernel_size=2, strides=1, padding='valid', dilation_rate=1))
model_exp7_1.add(layers.Activation("relu"))

model_exp7_1.add(layers.GlobalAveragePooling2D())
model_exp7_1.add(layers.Dense(64))
model_exp7_1.add(layers.Activation("relu"))
model_exp7_1.add(layers.Dense(3))
model_exp7_1.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_1, to_file='model_test_7.1.png', show_shapes=True)

epochs = 50

model_exp7_1.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_1.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 21. Code Snippet for Adding an Additional Block

On Fig. 21, there are now four convolutional layers, the specific approach researchers want to observe the effects of having an additional layer or block to the baseline CNN model. Since we have tried before the neuron size, now is time for an additional hidden layer of the neural network.

R. Experiment 7.2 / Adding an Additional Block 2 / Adding an Additional Block + Experiment 3 Changes

```
model_exp7_2 = Sequential()
model_exp7_2.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp7_2.add(layers.Rescaling(1.0 / 255))

model_exp7_2.add(layers.Conv2D(filters=8, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_2.add(layers.Activation("relu"))

model_exp7_2.add(layers.Conv2D(filters=16, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_2.add(layers.Activation("relu"))

model_exp7_2.add(layers.Conv2D(filters=32, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_2.add(layers.Activation("relu"))

model_exp7_2.add(layers.Conv2D(filters=64, kernel_size=2, strides=1, padding='valid', dilation_rate=1))
model_exp7_2.add(layers.Activation("relu"))

model_exp7_2.add(layers.GlobalAveragePooling2D())
model_exp7_2.add(layers.Dense(64))
model_exp7_2.add(layers.Activation("relu"))
model_exp7_2.add(layers.Dense(3))
model_exp7_2.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_2, to_file='model_test_7.2.png', show_shapes=True)

epochs = 50

model_exp7_2.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_2.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 22. Code Snippet for Adding an Additional Block with Experiment 3 Changes (Increased Number of Filters)

On Fig. 22, the 4 convolutional layers will also have each filtered doubled, the researchers seeks to find the effects of both increasing the number of filters and hidden layers combined and how it performs against the CNN baseline model.

S. Experiment 7.3 / Adding an Additional Block 3 / Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes

```
model_exp7_3 = Sequential()
model_exp7_3.add(keras.Input(shape=image_size + (3,))) # 64, 64, 3
model_exp7_3.add(layers.Rescaling(1.0 / 255))

model_exp7_3.add(layers.Conv2D(filters=8, kernel_size=32, strides=1, padding='valid', dilation_rate=1))
model_exp7_3.add(layers.Activation("relu"))

model_exp7_3.add(layers.Conv2D(filters=16, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_3.add(layers.Activation("relu"))

model_exp7_3.add(layers.Conv2D(filters=32, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_3.add(layers.Activation("relu"))

model_exp7_3.add(layers.Conv2D(filters=64, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_3.add(layers.Activation("relu"))

model_exp7_3.add(layers.GlobalAveragePooling2D())
model_exp7_3.add(layers.Dense(64))
model_exp7_3.add(layers.Activation("relu"))
model_exp7_3.add(layers.Dense(3))
model_exp7_3.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_3, to_file='model_test_7.3.png', show_shapes=True)

epochs = 50

model_exp7_3.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_3.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 23. Code Snippet for Adding an Additional Block with Experiment 3 Changes (Increased Number of Filters) and Experiment 4 Changes (Changing to Smaller Kernel Size)

On Fig. 23, the 4 convolutional layers combined with smaller kernel size, another distinct characteristic the researchers wanted to find out the value of having a different kernel size on the CNN model as well what may occur with an additional layer against the baseline CNN model.

T. Experiment 7.4 / Adding an Additional Block 4 / Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes

```
model_exp7_4 = Sequential()
model_exp7_4.add(keras.Input(shape=Image_size + (3,))) # 64, 64, 3
model_exp7_4.add(layers.Rescaling(1.0 / 255))

model_exp7_4.add(layers.Conv2D(filters=8, kernel_size=32, strides=1, padding='valid', dilation_rate=1))
model_exp7_4.add(layers.Activation("relu"))

model_exp7_4.add(layers.Conv2D(filters=16, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_4.add(layers.Activation("relu"))

model_exp7_4.add(layers.Conv2D(filters=32, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_4.add(layers.Activation("relu"))

model_exp7_4.add(layers.Conv2D(filters=64, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_4.add(layers.Activation("relu"))

model_exp7_4.add(layers.GlobalAveragePooling2D())
model_exp7_4.add(layers.Dense(128))
model_exp7_4.add(layers.Activation("relu"))
model_exp7_4.add(layers.Dense(3))
model_exp7_4.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_4, to_file='model_test_7_4.png', show_shapes=True)

epochs = 50

model_exp7_4.compile(
    optimizer=keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_4.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 24. Code Snippet for Adding an Additional Block with Experiment 3 Changes (Increased Number of Filters), Experiment 4 Changes (Changing to Smaller Kernel Size), and Experiment 6 Changes (Adding More Neurons)

The researchers keep adding modifications of previous experiments along with the additional 4th convolutional layers on which experiments 3,4, and 6 are applied can be seen on Fig. 24 on aim to further improve the performance of baseline CNN model.

U. Experiment 7.5 / Adding an Additional Block 5 / Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes + Experiment 5 Changes

```
model_exp7_5 = Sequential()
model_exp7_5.add(keras.Input(shape=Image_size + (3,))) # 64, 64, 3
model_exp7_5.add(layers.Rescaling(1.0 / 255))

model_exp7_5.add(layers.Conv2D(filters=8, kernel_size=32, strides=1, padding='valid', dilation_rate=1))
model_exp7_5.add(layers.Activation("relu"))

model_exp7_5.add(layers.Conv2D(filters=16, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_5.add(layers.Activation("relu"))

model_exp7_5.add(layers.Conv2D(filters=32, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_5.add(layers.Activation("relu"))

model_exp7_5.add(layers.Conv2D(filters=64, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_5.add(layers.Activation("relu"))

model_exp7_5.add(layers.GlobalAveragePooling2D())
model_exp7_5.add(layers.Dense(128))
model_exp7_5.add(layers.Activation("relu"))
model_exp7_5.add(layers.Dense(3))
model_exp7_5.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_5, to_file='model_test_7_5.png', show_shapes=True)

epochs = 50

model_exp7_5.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_5.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 25. Code Snippet for Adding an Additional Block with Experiment 3 Changes (Increased Number of Filters), Experiment 4 Changes (Changing to Smaller Kernel Size), Experiment 6 Changes (Adding More Neurons), and Experiment 5 Changes (Changing to a Smaller Learning Rate)

Almost close to the culmination of the hyperparameters done on the experiments 3,4, and 6 with the smaller learning rate on Fig. 25, the researchers ponder on the effects of these changes when compared against the baseline CNN model.

V. Experiment 7.6 / Adding an Additional Block 6 / Adding an Additional Block + Experiment 3 Changes + Experiment 6 Changes + Experiment 5 Changes + Experiment 2 Changes

```
model_exp7_6 = Sequential()
model_exp7_6.add(keras.Input(shape=Image_size + (3,))) # 64, 64, 3
model_exp7_6.add(layers.Rescaling(1.0 / 255))

model_exp7_6.add(layers.Conv2D(filters=8, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
model_exp7_6.add(layers.Activation("relu"))
model_exp7_6.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp7_6.add(layers.Conv2D(filters=16, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
model_exp7_6.add(layers.Activation("relu"))
model_exp7_6.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp7_6.add(layers.Conv2D(filters=32, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
model_exp7_6.add(layers.Activation("relu"))
model_exp7_6.add(layers.MaxPooling2D(pool_size=(2, 2)))

model_exp7_6.add(layers.Conv2D(filters=64, kernel_size=2, strides=1, padding='valid', dilation_rate=1))
model_exp7_6.add(layers.Activation("relu"))

model_exp7_6.add(layers.GlobalAveragePooling2D())
model_exp7_6.add(layers.Dense(128))
model_exp7_6.add(layers.Activation("relu"))
model_exp7_6.add(layers.Dense(3))
model_exp7_6.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(model_exp7_6, to_file='model_test_7_6.png', show_shapes=True)

epochs = 50

model_exp7_6.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
model_exp7_6.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 26. Code Snippet for Adding an Additional Block with Experiment 3 Changes (Increased Number of Filters), Experiment 6 Changes (Adding More Neurons), Experiment 5 Changes (Changing to a Smaller Learning Rate), and Experiment 2 Changes (Adding Max Pooling to each Convolutional Layer)

Fig. 26, almost close to the final experiments, the researchers applied experiment 2 changes where there is max pooling added to the end of each convolutional layer, so there are 4 max pooling layers at the current architecture. While applying the baseline CNN's kernel size as well.

W. Final CNN Architecture from Scratch

```
# Experiment 7.6 for better generalization
final_model = Sequential()
final_model.add(layers.Input(shape=image_size + (3,))) # 64, 64, 3

# Rescaling layer of Experiment 1.2
final_model.add(layers.Rescaling(scale=1.0/127.5, offset=-1))

final_model.add(layers.Conv2D(filters=8, kernel_size=16, strides=1, padding='valid', dilation_rate=1))
final_model.add(layers.Activation("relu"))
final_model.add(layers.MaxPooling2D(pool_size=(2, 2)))

final_model.add(layers.Conv2D(filters=16, kernel_size=8, strides=1, padding='valid', dilation_rate=1))
final_model.add(layers.Activation("relu"))
final_model.add(layers.MaxPooling2D(pool_size=(2, 2)))

final_model.add(layers.Conv2D(filters=32, kernel_size=4, strides=1, padding='valid', dilation_rate=1))
final_model.add(layers.Activation("relu"))
final_model.add(layers.MaxPooling2D(pool_size=(2, 2)))

final_model.add(layers.Conv2D(filters=64, kernel_size=2, strides=1, padding='valid', dilation_rate=1))
final_model.add(layers.Activation("relu"))

final_model.add(layers.GlobalAveragePooling2D())
final_model.add(layers.Dense(128))
final_model.add(layers.Activation("relu"))
final_model.add(layers.Dense(3))
final_model.add(layers.Activation("softmax"))

tf.keras.utils.plot_model(final_model, to_file='final_model.png', show_shapes=True)

epochs = 50

final_model.compile(
    optimizer=keras.optimizers.Adam(1e-4),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)
final_model.fit(train_ds, epochs=epochs, validation_data=val_ds)
```

Fig. 27. Code Snippet for the Final Scratch CNN Architecture including the combination of Experiment 7.6 for better Generalization and Experiment 1.2 Rescaling Layer

In the final experiment Fig. 27, the researchers applied to the baseline model the experiment 7.6 where there are 4 convolutional layers, doubled number of filters, smaller learning rate of 0.0001, and an additional max pooling layer to each convolutional layer. As well the rescaling approach on the experiment 1.2 on the preprocessing of the inputs. The researchers concluded that this is the best approach for the hair type multiclass classification based on the different experiments the researchers have conducted. Further discussion will be done on the results and analysis part of the paper.

X. Bonus Section : New Image Inputs of Anime Characters on Detecting Hair Type

Using the final model from Fig. 27, the researchers included new inputs besides the images in the hair_types folder. To add novelty, the researchers decided to add instead anime characters and determine their hair classification type. There are three new images of anime characters added in total.

```
img = keras.preprocessing.image.load_img(
    "anime_hair_types/straight_hair/straight_anime_girl1.jpg", target_size=image_size
)
img_array = keras.preprocessing.image.img_to_array(img)
img_array = tf.expand_dims(img_array, 0) # Create batch axis

# Display the image and the results
print("Image:")
display(Image(filename="anime_hair_types/straight_hair/straight_anime_girl1.jpg"))

predictions = final_model.predict(img_array)

print(
    "This image is %.2f percent curly hair, %.2f percent straight hair, and %.2f percent wavy hair."
    % tuple(predictions[0])
)
```

Fig. 28. Code Snippet of the Bonus Section



Fig. 29. Result of a Bonus Section Image

The image used is visualized and the hair classification output is given on Fig. 29, observed to be straight hair although the value of straight hair is 0.73 and the wavy hair is 0.25.

IV. RESULTS AND ANALYSIS

This section presents the outcomes of the conducted experiments and provides analysis on how specific architectural and hyperparameter modifications affected the CNN model's performance. The progression from the baseline to the final model involved systematic experimentation with data preprocessing, convolutional layers, filter sizes, learning rates, and the number of neurons. Performance was evaluated using various metrics such as Accuracy, Precision, Recall, F1-Score, Specificity, and ROC-AUC. Furthermore, input validation testing was performed to assess model behavior with respect to different hair types.

Baseline Model (B)

```
results_baseline_train = evaluate_model(model, train_ds, dataset_name="Baseline Model - Training")

===== Baseline Model - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.74       0.75       0.74       276
     1       0.59       0.60       0.59       252
     2       0.57       0.55       0.56       260

   accuracy       0.63
  macro avg       0.63
 weighted avg       0.63

Specificity (per class): [0.85546875 0.80223081 0.79356061]
ROC-AUC (macro average): 0.80880351848273675

results_baseline_val = evaluate_model(model, val_ds, dataset_name="Baseline Model - Validation")

===== Baseline Model - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.48       0.55       0.52        56
     1       0.48       0.47       0.47        70
     2       0.45       0.41       0.43        71

   accuracy       0.47
  macro avg       0.47
 weighted avg       0.47

Specificity (per class): [0.76595745 0.71653543 0.72222222]
ROC-AUC (macro average): 0.671496148637199
```

Fig. 30. Result of Baseline Model

From Fig. 30, the initial performance of the baseline model is described to be quite poor and the model is not confident on guessing the hair types yet with a large gap on training and validation accuracies with 60% and 47%, respectively. Suggesting that the model is underfitting and the researchers aims as the experiment progresses to see both accuracy values increase and the loss values decrease.

A. Individual Performances

Different Image Rescaling with Mean and Variance (Experiment 1.1)

```
results_model_exp1_1_train = evaluate_model(model_exp1_1, train_ds, dataset_name="Experiment 1.1 - Training")

===== Experiment 1.1 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.35       1.00       0.52       276
     1       0.71       0.02       0.04       252
     2       0.00       0.00       0.00       260

   accuracy       0.36
  macro avg       0.34
 weighted avg       0.35

Specificity (per class): [0.01171875 0.99626866 1.         ]
ROC-AUC (macro average): 0.5067723142089659
```

Fig. 31. Training Evaluation Result of Image Rescaling with Mean and Variance

```
results_model_exp1_1_val = evaluate_model(model_exp1_1, val_ds, dataset_name="Experiment 1.1 - Validation")

===== Experiment 1.1 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.28       0.98       0.44        56
     1       0.00       0.00       0.00        70
     2       0.00       0.00       0.00        71

   accuracy       0.09
  macro avg       0.09
 weighted avg       0.08

Specificity (per class): [0.         0.99212598 1.         ]
ROC-AUC (macro average): 0.5018801862987693
```

Fig. 32. Validation Evaluation Result of Image Rescaling with Mean and Variance

From Fig. 31 and Fig. 32, the performance after changing rescaling to mean equal to zero and variance is equal to one, the performance on all performance metrics is considered to be insignificant, the accuracies are also closely to guessing of a 1/3 chance or 33%, therefore researchers must avoid using the specific approach at any costs.

Scaling to the Range [-1, 1] (Experiment 1.2)

```
results_model_exp1_2_train = evaluate_model(model_exp1_2, train_ds, dataset_name="Experiment 1.2 - Training")

===== Experiment 1.2 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.74       0.88       0.81       276
     1       0.67       0.56       0.61       252
     2       0.64       0.62       0.63       260

   accuracy       0.69
  macro avg       0.69
 weighted avg       0.69

Specificity (per class): [0.83789062 0.86940299 0.82954545]
ROC-AUC (macro average): 0.8649579449899035

results_model_exp1_2_val = evaluate_model(model_exp1_2, val_ds, dataset_name="Experiment 1.2 - Validation")

===== Experiment 1.2 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.60       0.71       0.65        56
     1       0.57       0.51       0.54        70
     2       0.54       0.51       0.52        71

   accuracy       0.57
  macro avg       0.57
 weighted avg       0.57

Specificity (per class): [0.80851064 0.78740157 0.75396825]
ROC-AUC (macro average): 0.7489500416558409
```

Fig. 33. Result of Scaling to Range [-1, 1]

From Fig. 33, the performance after changing rescaling to a zero-centered value has a better performance compared to the baseline CNN model. Metrics such as accuracy on both training and validation, such as 0.63 and 0.47, respectively on the baseline while increased 0.69 and 0.57 values on the experiment 1.2, respectively. Precision as well that is 0.63 and 0.47 training and validation values on baseline, while on the experiment 1.2 is 0.69 and 0.57, respectively. Recall and F1-score also follows the trend where the experiment 1.2 values performed better than the baseline CNN mode. Thus, it is expected in future experiments that this approach will be combined with other experiments.

Using Integer Lookup for Scaling (Experiment 1.3)

```
results_model_exp1_3_train = evaluate_model(model_exp1_3, train_ds_scaled_lookup, dataset_name="Experiment 1.3 - Training")

===== Experiment 1.3 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.62       0.78       0.69       276
     1       0.56       0.51       0.53       252
     2       0.51       0.41       0.45       260

   accuracy       0.56
  macro avg       0.57
 weighted avg       0.56

Specificity (per class): [0.74802438 0.81156716 0.8038903 ]
ROC-AUC (macro average): 0.7448095189884724

results_model_exp1_3_val = evaluate_model(model_exp1_3, val_ds_scaled_lookup, dataset_name="Experiment 1.3 - Validation")

===== Experiment 1.3 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.48       0.64       0.58        56
     1       0.48       0.39       0.43        70
     2       0.48       0.38       0.34        71

   accuracy       0.43
  macro avg       0.44
 weighted avg       0.43

Specificity (per class): [0.62411348 0.77165354 0.75396825]
ROC-AUC (macro average): 0.6553599416157786
```

Fig. 34. Result of Using Integer Lookup for Scaling

From Fig. 34, to briefly conclude the different preprocessing experiments from 1.1 to 1.3, the experiment 1.2 performed better against the current 1.3 experiment where using integer lookup for scaling performed below the baseline model. Therefore, discarding the approach for future experiments is decided by researchers.

Regarding analysis of experiment 1, experiment 1.1 underperformed, weak, and ineffective to be used while experiment 1.3 is too overconfident on prediction and also has lower accuracy similar to 1.1. Thus, Experiment 1.2 has the best performance and can do a better generalization compared although there are still signs of overfitting where training accuracy is still significantly higher than validation accuracy.

Adding Max Pooling 2D on each Convolutional Layer (Experiment 2)

```

results_model_exp2_train = evaluate_model(model_exp2, train_ds, dataset_name="Experiment 2 - Training")

===== Experiment 2 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.65       0.61       0.63       276
     1       0.67       0.74       0.70       252
     2       0.69       0.65       0.67       260

   accuracy       0.74       0.74       0.74       788
  macro avg       0.74       0.74       0.74       788
 weighted avg       0.74       0.74       0.74       788

Specificity (per class): [0.921875  0.82835821 0.85795455]
ROC-AUC (macro average): 0.8973684264492348

results_model_exp2_val = evaluate_model(model_exp2, val_ds, dataset_name="Experiment 2 - Validation")

===== Experiment 2 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.55       0.52       0.53        56
     1       0.56       0.60       0.58        70
     2       0.58       0.56       0.57        71

   accuracy       0.56       0.56       0.56       197
  macro avg       0.56       0.56       0.56       197
 weighted avg       0.56       0.56       0.56       197

Specificity (per class): [0.82978723 0.74015748 0.76984127]
ROC-AUC (macro average): 0.7484151842152595

```

Fig. 35. Result of Adding Max Pooling on each Convolutional Layer

From Fig. 35, the training performance of the experiment 2 of adding a max pooling layer after each convolution improved. Despite the increase take the example where the validation accuracies of experiment 1.2 vs experiment 2, 0.57 and 0.56, respectively. These two experiments almost have the similar performance and above the baseline CNN mode, therefore this interests the researchers to combine with other experiments in the later attempts when combining different experiments.

Comments include experiment 2 is currently the best model against other previous models conducted, that adding max pooling enhanced the model's capacity to generalize while reducing the risk of overfitting.

Doubling the Filters for each convolutional layer (8,16,32) (Experiment 3)

```

results_model_exp3_train = evaluate_model(model_exp3, train_ds, dataset_name="Experiment 3 - Training")

===== Experiment 3 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.78       0.56       0.65       276
     1       0.48       0.73       0.58       252
     2       0.44       0.35       0.39       260

   accuracy       0.57       0.55       0.54       788
  macro avg       0.57       0.55       0.54       788
 weighted avg       0.57       0.55       0.54       788

Specificity (per class): [0.9148625  0.625  0.78598485]
ROC-AUC (macro average): 0.7642643115461913

results_model_exp3_val = evaluate_model(model_exp3, val_ds, dataset_name="Experiment 3 - Validation")

===== Experiment 3 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.59       0.39       0.47        56
     1       0.48       0.57       0.52        70
     2       0.39       0.42       0.41        71

   accuracy       0.49       0.46       0.47       197
  macro avg       0.49       0.46       0.47       197
 weighted avg       0.48       0.47       0.47       197

Specificity (per class): [0.89361782 0.65354331 0.63492863]
ROC-AUC (macro average): 0.6686389827562743

```

Fig. 36. Result of Doubling the Number of Filters for each Convolutions

From Fig. 36, the metric results such as validation accuracy, precision, recall, and f1-score of experiment 3 are similar to the baseline CNN Model. The researchers seek to find improvement further by combining with the other experiments such as experiment 1.2 to further improve on achieving the final model.

Comments include that experiment 3 underperformed significantly due to too many filters early on while not having pooling layers to reduce the complexity of the model resulting in vanishing of the gradients or ineffective model.

Changing to a Smaller Kernel Size (3,3) (Experiment 4)

```

results_model_exp4_train = evaluate_model(model_exp4, train_ds, dataset_name="Experiment 4 - Training")

===== Experiment 4 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.55       0.73       0.63       276
     1       0.52       0.32       0.40       252
     2       0.54       0.55       0.54       260

   accuracy       0.54       0.53       0.54       788
  macro avg       0.54       0.53       0.53       788
 weighted avg       0.54       0.54       0.53       788

Specificity (per class): [0.67773438 0.86007463 0.76893939]
ROC-AUC (macro average): 0.7242469943267608

results_model_exp4_val = evaluate_model(model_exp4, val_ds, dataset_name="Experiment 4 - Validation")

===== Experiment 4 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.35       0.61       0.44        56
     1       0.53       0.26       0.35        70
     2       0.48       0.45       0.47        71

   accuracy       0.45       0.44       0.43       197
  macro avg       0.45       0.44       0.42       197
 weighted avg       0.46       0.43       0.42       197

Specificity (per class): [0.55319149 0.87401575 0.73015873]
ROC-AUC (macro average): 0.6116875758001031

```

Fig. 37. Result of Changing to a Smaller Kernel Size

From Fig.37, comparing the results to the baseline model, they are underperforming below the baseline model.

Thus, researchers will also attempt combining different features also with the specific experiment and monitor whether improvements occurred on the combined experiments models such on experiment 7.

Comments for analysis of experiment 4 includes a modest improvement compared to the baseline but still underperforms. Benefit from smaller kernels but still lacks pooling that is the key for the dataset and may also benefit from BatchNorm or dropout.

Changing Learning Rate from 0.001 to 0.0001 (Experiment 5)

```
results_model_exp5_train = evaluate_model(model_exp5, train_ds, dataset_name="Experiment 5 - Training")

===== Experiment 5 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.59       0.75       0.66       276
     1       0.55       0.35       0.43       252
     2       0.47       0.50       0.49       260

 accuracy         0.54         0.54         0.54       788
  macro avg       0.54         0.53         0.53       788
 weighted avg     0.54         0.54         0.53       788

Specificity (per class): [0.72070312 0.86753731 0.71969697]
ROC-AUC (macro average): 0.725286660514183

results_model_exp5_val = evaluate_model(model_exp5, val_ds, dataset_name="Experiment 5 - Validation")

===== Experiment 5 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.44       0.62       0.51        56
     1       0.49       0.31       0.38        70
     2       0.44       0.45       0.45        71

 accuracy         0.46         0.46         0.45       197
  macro avg       0.46         0.45         0.45       197
 weighted avg     0.46         0.45         0.44       197

Specificity (per class): [0.68085106 0.81889764 0.68253968]
ROC-AUC (macro average): 0.662126083567793
```

Fig. 38. Result of Changing Learning Rate from 1e-3 to 1e-4

From Fig. 38, changing the learning rate from 0.001 to 0.0001 have the performance of the CNN model perform below the baseline across different metrics such as validation accuracy 0.47vs. 0.45, precision 0.47 vs. 0.46 but the effect is not significant when changing only one hyperparameter, itself.

Comments on analyzing experiment 5 include the lower learning rate helps the training to stabilize but the model seems to underfit, suggested by the lower training accuracy even at 50 epochs. Thus, performance is identical to experiment 4 , where pooling may have a bigger missing piece than kernel size or learning rate alone.

Adding More Neurons (Experiment 6)

```
results_model_exp6_train = evaluate_model(model_exp6, train_ds, dataset_name="Experiment 6 - Training")

===== Experiment 6 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.66       0.74       0.70       276
     1       0.64       0.40       0.50       252
     2       0.49       0.60       0.54       260

 accuracy         0.59         0.59         0.59       788
  macro avg       0.60         0.58         0.58       788
 weighted avg     0.60         0.59         0.58       788

Specificity (per class): [0.7966875  0.89179104 0.68939394]
ROC-AUC (macro average): 0.779490185420125

results_model_exp6_val = evaluate_model(model_exp6, val_ds, dataset_name="Experiment 6 - Validation")

===== Experiment 6 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.51       0.64       0.57        56
     1       0.62       0.34       0.44        70
     2       0.48       0.59       0.53        71

 accuracy         0.52         0.52         0.52       197
  macro avg       0.54         0.53         0.51       197
 weighted avg     0.54         0.52         0.51       197

Specificity (per class): [0.75886525 0.88188976 0.63492063]
ROC-AUC (macro average): 0.6853434899900325
```

Fig. 39. Result of Adding More Neurons

From Fig. 39, adding more neurons when compared to the baseline model, significantly improving validation accuracy from 0.47 to 0.52, validation recall from 0.47 to 0.52 as well. Close to the final experiments, the experiment 6 will also undergo testing with the experiments previously conducted.

Additional comments regarding experiment 6 is that if the features to be extracted are not that good then increasing the dense layer size will not help. Thus, no positive results against the baseline model as the early convolutional layers were not extracting useful features.

Adding an Additional Block (Experiment 7.1)

```
results_model_exp7_1_train = evaluate_model(model_exp7_1, train_ds, dataset_name="Experiment 7.1 - Training")

===== Experiment 7.1 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.61       0.86       0.71       276
     1       0.57       0.39       0.46       252
     2       0.50       0.44       0.47       260

 accuracy         0.56         0.56         0.57       788
  macro avg       0.56         0.56         0.55       788
 weighted avg     0.56         0.57         0.55       788

Specificity (per class): [0.70898438 0.86007463 0.78030303]
ROC-AUC (macro average): 0.7702235554982161

results_model_exp7_1_val = evaluate_model(model_exp7_1, val_ds, dataset_name="Experiment 7.1 - Validation")

===== Experiment 7.1 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.38       0.71       0.50        56
     1       0.39       0.23       0.29        70
     2       0.43       0.31       0.36        71

 accuracy         0.40         0.42         0.40       197
  macro avg       0.40         0.42         0.38       197
 weighted avg     0.40         0.40         0.37       197

Specificity (per class): [0.53900709 0.80314961 0.76984127]
ROC-AUC (macro average): 0.6568053877767327
```

Fig. 40. Results of an Additional Convolutional Block

From Fig. 40, when compared to the baseline model underperformed and struggled to generalize. Adding depth did

not help improving the performance without balancing capacity or regularization.

Adding an Additional Block + Experiment 3 Changes + (Experiment 7.2)

```
results_model_exp7_2_train = evaluate_model(model_exp7_2, train_ds, dataset_name="Experiment 7.2 - Training")

===== Experiment 7.2 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.74      0.71      0.72        276
     1       0.70      0.46      0.56        252
     2       0.54      0.74      0.63        260

   accuracy      0.66      0.64      0.64        788
  macro avg      0.66      0.64      0.64        788
 weighted avg      0.66      0.64      0.64        788

Specificity (per class): [0.86132812 0.90858209 0.69318182]
ROC-AUC (macro average): 0.8103621505419306

results_model_exp7_2_val = evaluate_model(model_exp7_2, val_ds, dataset_name="Experiment 7.2 - Validation")

===== Experiment 7.2 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.55      0.55      0.55         56
     1       0.61      0.40      0.48         70
     2       0.53      0.70      0.60         71

   accuracy      0.56      0.55      0.55        197
  macro avg      0.56      0.55      0.55        197
 weighted avg      0.56      0.55      0.55        197

Specificity (per class): [0.82269504 0.85826772 0.64285714]
ROC-AUC (macro average): 0.7123466355487902
```

Fig. 41. Results of an Adding an Additional Block + Experiment 3 Changes

From Fig. 41, when compared to the baseline model improved where increasing channel depth allowed the network to learn more features, helping in both training and validation performance.

Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes (Experiment 7.3)

```
results_model_exp7_3_train = evaluate_model(model_exp7_3, train_ds, dataset_name="Experiment 7.3 - Training")

===== Experiment 7.3 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.35      1.00      0.52        276
     1       0.00      0.00      0.00        252
     2       0.00      0.00      0.00        260

   accuracy      0.12      0.33      0.17        788
  macro avg      0.12      0.33      0.16        788
 weighted avg      0.12      0.35      0.18        788

Specificity (per class): [0. 1. 1.]
ROC-AUC (macro average): 0.5
```

Fig. 42. Training Evaluation Results Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes

```
results_model_exp7_3_val = evaluate_model(model_exp7_3, val_ds, dataset_name="Experiment 7.3 - Validation")

===== Experiment 7.3 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.28      1.00      0.44         56
     1       0.00      0.00      0.00         70
     2       0.00      0.00      0.00         71

   accuracy      0.09      0.33      0.15        197
  macro avg      0.09      0.33      0.15        197
 weighted avg      0.08      0.28      0.13        197

Specificity (per class): [0. 1. 1.]
ROC-AUC (macro average): 0.5
```

Fig. 43. Validation Evaluation Results of Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes

From Fig. 42 and Fig. 43, when compared to the baseline model underperformed drastically with an accuracy of only 0.35 and 0.28 for the training and validation values,

which is close to comparing that the model is just guessing and getting it correct 1/3 of the time which is not good.

Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes (Experiment 7.4)

```
results_model_exp7_4_train = evaluate_model(model_exp7_4, train_ds, dataset_name="Experiment 7.4 - Training")

===== Experiment 7.4 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.35      1.00      0.52        276
     1       0.00      0.00      0.00        252
     2       0.00      0.00      0.00        260

   accuracy      0.12      0.33      0.17        788
  macro avg      0.12      0.33      0.16        788
 weighted avg      0.12      0.35      0.18        788
```

Fig. 44. Training Evaluation Results of Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes

```
results_model_exp7_4_val = evaluate_model(model_exp7_4, val_ds, dataset_name="Experiment 7.4 - Validation")

===== Experiment 7.4 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.35      1.00      0.52         56
     1       0.00      0.00      0.00         70
     2       0.00      0.00      0.00         71

   accuracy      0.12      0.33      0.17        197
  macro avg      0.12      0.33      0.16        197
 weighted avg      0.12      0.35      0.18        197
```

Fig. 45. Validation Evaluation Result of Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes

From Fig. 44 and Fig. 45, performed similar to experiment 7.3 which is close to comparing that the model is just guessing and not an acceptable standard.

Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes + Experiment 5 Changes (Experiment 7.5)

```
results_model_exp7_5_train = evaluate_model(model_exp7_5, train_ds, dataset_name="Experiment 7.5 - Training")

===== Experiment 7.5 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.72      0.93      0.81        276
     1       0.76      0.62      0.68        252
     2       0.73      0.63      0.68        260

   accuracy      0.73      0.73      0.73        788
  macro avg      0.73      0.73      0.73        788
 weighted avg      0.73      0.73      0.73        788

Specificity (per class): [0.80664062 0.90671642 0.88257576]
ROC-AUC (macro average): 0.8947363948026502

results_model_exp7_5_val = evaluate_model(model_exp7_5, val_ds, dataset_name="Experiment 7.5 - Validation")

===== Experiment 7.5 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.47      0.73      0.57         56
     1       0.63      0.41      0.50         70
     2       0.62      0.56      0.59         71

   accuracy      0.58      0.57      0.56        197
  macro avg      0.58      0.57      0.56        197
 weighted avg      0.58      0.56      0.55        197

Specificity (per class): [0.67375887 0.86614173 0.80952381]
ROC-AUC (macro average): 0.7592966044437732
```

Fig. 46. Results of Adding an Additional Block + Experiment 3 Changes + Experiment 4 Changes + Experiment 6 Changes + Experiment 5 Changes

From Fig. 46, has a huge boost when compared to different experiments on section 7 of the experiment, whereas

lowering the learning rate allowed the model to converge effectively and stably.

Adding an Additional Block + Experiment 3 Changes + Experiment 6 Changes + Experiment 5 Changes + Experiment 2 Changes (Experiment 7.6)

```

results_model_exp7_6_train = evaluate_model(model_exp7_6, train_ds, dataset_name="Experiment 7.6 - Training")

===== Experiment 7.6 - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.72       0.86       0.79       276
     1       0.79       0.60       0.68       252
     2       0.68       0.70       0.69       260

   accuracy       0.73
  macro avg       0.72
 weighted avg       0.72

Specificity (per class): [0.82226562 0.92537313 0.83522727]
ROC-AUC (macro average): 0.885003845097135

results_model_exp7_6_val = evaluate_model(model_exp7_6, val_ds, dataset_name="Experiment 7.6 - Validation")

===== Experiment 7.6 - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.51       0.71       0.60        56
     1       0.68       0.49       0.57        70
     2       0.61       0.59       0.60        71

   accuracy       0.60
  macro avg       0.60
 weighted avg       0.61

Specificity (per class): [0.73049645 0.87401575 0.78571429]
ROC-AUC (macro average): 0.7780644496902674

```

Fig. 47. Additional Block + Experiment 3 Changes + Experiment 6 Changes + Experiment 5 Changes + Experiment 2 Changes

From Fig. 47, the model has improved validation accuracy further and reduced the overfitting. Whereas pooling helped preserve important spatial information while reducing the dimensions of the features.

Final Scratch CNN Architecture (F)

```

results_final_model_train = evaluate_model(final_model, train_ds, dataset_name="Final Model - Training")

===== Final Model - Training Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.91       0.95       0.93       276
     1       0.83       0.92       0.87       252
     2       0.93       0.77       0.84       260

   accuracy       0.89
  macro avg       0.88
 weighted avg       0.88

Specificity (per class): [0.94726562 0.90858209 0.96969697]
ROC-AUC (macro average): 0.975061145330077

results_final_model_val = evaluate_model(final_model, val_ds, dataset_name="Final Model - Validation")

===== Final Model - Validation Evaluation =====
Classification Report:
      precision    recall  f1-score   support

     0       0.62       0.66       0.64        56
     1       0.56       0.70       0.62        70
     2       0.70       0.49       0.58        71

   accuracy       0.63
  macro avg       0.62
 weighted avg       0.61

Specificity (per class): [0.83687943 0.7007874 0.88095238]
ROC-AUC (macro average): 0.8012240141643469

```

Fig. 48. Additional Block + Experiment 3 Changes + Experiment 6 Changes + Experiment 5 Changes + Experiment 2 Changes

From Fig. 48, The final model achieved the best performance compared to all previous experiments whereas the model demonstrated high training accuracy and strong precision performance indicated that the model effectively

learned to distinguish the different hair types. However, the validation dropped significantly where there is a prevailing issue regarding the ability of the model to generalize. On ROC-AUC, the specificity was comparably higher than in validation, interpreting that the model likely overfitted despite the enhancements from previous experiments such as adding global average pooling and smaller convolutional filters.

Additional comments regard that the model performed well on training but became limited on the validation set. To address the issue, researchers suggest incorporating regularization techniques such as enhanced data augmentation or simplifying the model.

B. Comparison of Experimented Models

TABLE 5
Accuracy of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training	Validation
B	Baseline	0.63	0.47
1.1	Rescaling using: mean=0 variance=1	0.36	0.28
1.2	Rescaling using: Range [-1, 1]	0.69	0.57
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.57	0.43
2	Max Pooling 2D (2, 2)	0.74	0.56
3	Filters (8, 16, 32)	0.55	0.47
4	Kernel (4, 8, 16)	0.54	0.43
5	Learning Rate: 1e-4	0.54	0.45
6	Neurons 64->128	0.59	0.52
7.1	+1 Convolutionary Block	0.57	0.40
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.64	0.55
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	0.35	0.28
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	0.35	0.28
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.73	0.56
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.72	0.59
F	Final Model Rescaling using: Range [-1, 1] +1 Convolutionary Block	0.88	0.61

	Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4		
--	--	--	--

Accuracy (Table 5): The baseline model achieved a training accuracy of 0.63 and a validation accuracy of 0.47. The most significant improvement was observed in the final model, which used a combination of range normalization [-1, 1], an added convolutional block, filters (8, 16, 32), increased neurons (64→128), and a learning rate of 1e-4. This configuration achieved 0.88 training and 0.61 validation accuracy. Notably, range-based normalization and convolutional expansion played crucial roles in boosting accuracy.

TABLE 6

Precision of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training	Validation
B	Baseline	0.63	0.47
1.1	Rescaling using: mean=0 variance=1	0.35	0.08
1.2	Rescaling using: Range [-1, 1]	0.69	0.57
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.56	0.43
2	Max Pooling 2D (2, 2)	0.74	0.56
3	Filters (8, 16, 32)	0.57	0.48
4	Kernel (4, 8, 16)	0.54	0.46
5	Learning Rate: 1e-4	0.54	0.46
6	Neurons 64->128	0.60	0.54
7.1	+1 Convolutionary Block	0.56	0.40
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.66	0.56
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	0.12	0.08
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	0.12	0.08
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.73	0.58
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.73	0.61
F	Final Model Rescaling using: Range [-1, 1] +1 Convolutionary Block	0.89	0.63

	Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4		
--	--	--	--

TABLE 7

Recall of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training	Validation
B	Baseline	0.63	0.47
1.1	Rescaling using: mean=0 variance=1	0.36	0.28
1.2	Rescaling using: Range [-1, 1]	0.69	0.57
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.57	0.43
2	Max Pooling 2D (2, 2)	0.74	0.56
3	Filters (8, 16, 32)	0.55	0.47
4	Kernel (4, 8, 16)	0.54	0.43
5	Learning Rate: 1e-4	0.54	0.45
6	Neurons 64->128	0.59	0.52
7.1	+1 Convolutionary Block	0.57	0.40
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.64	0.55
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	0.35	0.28
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	0.35	0.28
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.73	0.56
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.59	0.59
F	Final Model Rescaling using: Range [-1, 1] +1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.88	0.61

Precision (Table 6) and Recall (Table 7) followed similar trends, with the final model attaining validation scores of 0.63 and 0.61, respectively, compared to the baseline 0.47. This confirms that the final model effectively reduced false positives and false negatives. Additionally, techniques such as MaxPooling and appropriate filter scaling significantly improved these metrics.

TABLE 8

F1-Score of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training	Validation
B	Baseline	0.63	0.47
1.1	Rescaling using: mean=0 variance=1	0.19	0.12
1.2	Rescaling using: Range [-1, 1]	0.69	0.57
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.56	0.42
2	Max Pooling 2D (2, 2)	0.74	0.56
3	Filters (8, 16, 32)	0.54	0.47
4	Kernel (4, 8, 16)	0.53	0.42
5	Learning Rate: 1e-4	0.53	0.44
6	Neurons 64->128	0.58	0.51
7.1	+1 Convolutionary Block	0.55	0.37
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.64	0.55
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	0.18	0.13
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	0.18	0.13
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.73	0.55
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.60	0.59
F	Final Model Rescaling using: Range [-1, 1] +1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.88	0.61

F1-Score (Table 8), a balanced harmonic mean of precision and recall, exhibited the same upward trend—reaching 0.61 in validation compared to 0.47 for the baseline. The poorest performing model applied normalization with mean=0 and variance=1, showing clear evidence that such preprocessing was detrimental across all metrics.

TABLE 9

Specificity of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training			Validation		
		Curly	Straight	Wavy	Curly	Straight	Wavy
B	Baseline	0.85	0.80	0.79	0.76	0.71	0.72
1.1	Rescaling using: mean=0 variance=1	(computation error)					
1.2	Rescaling using: Range [-1, 1]	0.83	0.86	0.82	0.80	0.78	0.75
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.74	0.81	0.80	0.62	0.77	0.75
2	Max Pooling 2D (2, 2)	0.92	0.82	0.85	0.82	0.74	0.76
3	Filters (8, 16, 32)	0.91	0.62	0.78	0.89	0.65	0.63
4	Kernel (4, 8, 16)	0.67	0.86	0.76	0.55	0.87	0.73
5	Learning Rate: 1e-4	0.72	0.86	0.71	0.68	0.81	0.68
6	Neurons 64->128	0.79	0.89	0.68	0.75	0.88	0.63
7.1	+1 Convolutionary Block	0.70	0.86	0.78	0.53	0.80	0.76
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.86	0.90	0.69	0.82	0.85	0.64
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	(computation error)					
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	(computation error)					
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.80	0.90	0.88	0.67	0.86	0.80
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.82	0.92	0.83	0.73	0.87	0.78

F	Final Model Rescaling using: Range [-1, 1] +1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.94	0.90	0.96	0.83	0.70	0.88
---	--	-------------	------	-------------	-------------	------	-------------

Specificity (Table 9) revealed the model's ability to correctly identify negative classes. The final model achieved high specificity across all hair types, particularly for wavy hair (0.88 validation score), demonstrating its improved discriminatory power in multi-class classification.

TABLE 10
ROC-AUC of baseline model, the individual modifications of baseline model, and the final model

#	Experiment	Training	Validation
B	Baseline	0.80	0.67
1.1	Rescaling using: mean=0 variance=1	0.50	0.50
1.2	Rescaling using: Range [-1, 1]	0.86	0.74
1.3	Rescaling using: Integer Lookup for faster Rescaling	0.74	0.65
2	Max Pooling 2D (2, 2)	0.89	0.74
3	Filters (8, 16, 32)	0.76	0.66
4	Kernel (4, 8, 16)	0.72	0.61
5	Learning Rate: 1e-4	0.72	0.66
6	Neurons 64->128	0.77	0.68
7.1	+1 Convolutionary Block	0.77	0.65
7.2	+1 Convolutionary, Block Filters(8, 16, 32)	0.81	0.71
7.3	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16)	0.50	0.50
7.4	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128	0.50	0.50
7.5	+1 Convolutionary Block Filters(8, 16, 32) Kernel (4, 8, 16) Neurons 64->128 Learning Rate: 1e-4	0.89	0.75
7.6	+1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4	0.88	0.77
F	Final Model: Rescaling using: Range [-1,	0.97	0.80

1] +1 Convolutionary Block Filters(8, 16, 32) Neurons 64->128 Learning Rate: 1e-4		
---	--	--

ROC-AUC (Table 10) metrics provide insight into the model's performance across different thresholds. The final model achieved the highest validation ROC-AUC score of 0.80, indicating reliable overall classification performance. Baseline performance stood at 0.67, while certain failed configurations (e.g., poor normalization) yielded scores as low as 0.50.

Table 11
Input Validation

Models	Curvy Hair	Straight Hair	Wavy Hair
Baseline	C: 0.85 S: 0.14 W: 0.02	C: 0.68 S: 0.22 W: 0.10	C: 0.03 S: 0.48 W: 0.49
Experiment Model 1.1	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33
Experiment Model 1.2	C: 0.98 S: 0.02 W: 0.00	C: 0.49 S: 0.23 W: 0.28	C: 0.00 S: 0.42 W: 0.58
Experiment Model 1.3	C: 1 S: 0 W: 0	C: 1 S: 0 W: 0	C: 0.10 S: 0.51 W: 0.39
Experiment Model 2	C: 0.98 S: 0.01 W: 0	C: 0.30 S: 0.56 W: 0.14	C: 0.50 S: 0.28 W: 0.22
Experiment Model 3	C: 0.65 S: 0.24 W: 0.10	C: 0.40 S: 0.23 W: 0.37	C: 0.05 S: 0.62 W: 0.33
Experiment Model 4	C: 0.89 S: 0.09 W: 0.02	C: 0.47 S: 0.29 W: 0.24	C: 0.19 S: 0.39 W: 41
Experiment Model 5	C: 0.71 S: 0.20 W: 0.09	C: 0.60 S: 0.20 W: 0.20	C: 0.16 S: 0.39 W: 0.45
Experiment Model 6	C: 0.87 S: 0.10 W: 0.03	C: 0.54 S: 0.13 W: 0.33	C: 0.04 S: 0.58 W: 0.38
Experiment Model 7.1	C: 0.83 S: 0.15 W: 0.02	C: 0.68 S: 0.28 W: 0.04	C: 0.47 S: 0.24 W: 0.30
Experiment Model 7.2	C: 0.69 S: 0.28 W: 0.04	C: 0.35 S: 0.23 W: 0.42	C: 0.01 S: 0.76 W: 0.22
Experiment Model 7.3	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33
Experiment Model 7.4	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33	C: 0.35 S: 0.32 W: 0.33

Experiment Model 7.5	C: 1	C: 0.03	C: 0.90
	S: 0	S: 0.30	S: 0.01
	W: 0	W: 0.67	W: 0.09
Experiment Model 7.6	C: 0.91	C: 0.26	C: 0.69
	S: 0.06	S: 0.53	S: 0.17
	W: 0.03	W: 0.22	W: 0.14
Final Model	C: 0.90	C: 0.63	C: 0.01
	S: 0.09	S: 0.18	S: 0.77
	W: 0.01	W: 0.19	W: 0.23



Fig. 49. Input Image for Curly Hair



Fig. 50. Input Image for Straight Hair



Fig. 51. Input Image for Wavy Hair

To assess the model's generalization and robustness, a set of real-world images representing three distinct hair types—Curly, Straight, and Wavy—were used for input validation testing, as shown in Figures 30–32. The classification distribution across different models is presented in Table 10.

The baseline model showed a strong bias toward the Curly class, often misclassifying Straight and Wavy hair. For instance, in the Curly Hair image, the model predicted 85% Curly, but only 14% and 2% for Straight and Wavy, respectively.

Experiment Models 1.3 and 7.5 severely overfit the training data, classifying Curly Hair with 100% certainty and completely disregarding other classes—evidence of poor generalization.

The final model, however, achieved a more balanced classification:

For Curly hair, 90% Curly was predicted correctly, with minimal confusion between Straight (9%) and Wavy (1%). For Straight hair, 63% Straight, 18% Wavy, and 19% Curly were predicted—demonstrating better separation of features. For Wavy hair, while 77% was correctly classified, some confusion remained with Straight (23%).

V. CONCLUSIONS

The conclusions section is the summary of the experiment conducted and gives spotlight on some of the key highlights that were remarkable and insightful for the researchers.

The research effectively tested the use of a Convolutional Neural Network (CNN) to classify hair types as curly, straight, and wavy through the systematic alteration of hyperparameters and testing their effect on model performance. Among the findings, it was observed that there was a critical effect of preprocessing methods, such that rescaling images to the range $[-1,1]$ performed better than others at a validation accuracy rate of 0.57 compared to 0.28 through normalization with mean=0 and variance=1. This enhancement underscores the benefit of zero-centered inputs for model convergence and feature extraction. Architectural changes, including the inclusion of max pooling layers and network extension with an extra convolutional block, also improved performance, with the latter accounting for a significant training accuracy of 0.88. Hyperparameter tuning, specifically lowering the learning rate to $1e-4$, was instrumental in stabilizing training and enhancing validation metrics, as seen by the ROC-AUC score of 0.77 over the baseline 0.67. The final model, which integrated these optimizations, had a training accuracy of 0.88 and a validation accuracy of 0.61, showing its potential for practical use. Nevertheless, difficulties in differentiating straight and curly hair indicate avenues for future optimization, including higher

regularization or greater datasets. Overall, the work emphasizes the value of empirical hyperparameter tuning and offers insightful observations for future AI-based beauty and fashion research.

Recommendations of the researchers includes exploring other hyperparameters that the researchers weren't able to tackle such as dilation and stride. Exploring these hyperparameters may give future studies new opportunities on how they may tackle and approach building a better model against the researchers' concluded final model. Also, researchers also suggest the use of alternatives besides Google Collab application, researchers faced some issues such as they weren't able to use their own GPU resource to process the CNN feature extraction that is indeed expensive and usually take around an hour to complete the whole experiment process. Thus resorting to the available GPU option on the application but also being available on a limited time. Lastly, the researchers weren't able to visually provide the accuracy graph of training versus the validation accuracy performance due to needing a third-party app called "graphviz" that researchers attempted to fix and install but weren't successful in making it work.

REFERENCES

- [1] Jabraeili, D. M. (2025). The application of artificial intelligence in the beauty industry: A technological revolution in the world of makeup and skincare . International Journal of Modern Achievement in Science, Engineering and Technology, 2(1), 156–158. <https://doi.org/10.63053/ijset.71>
- [2] Laxminarayana, C. (2025, April 14). Council Post: How AI is Reshaping Corporate Decision-Making: From Data to Insights. Forbes. <https://www.forbes.com/councils/forbestechcouncil/2025/04/14/how-ai-is-reshaping-corporate-decision-making-from-data-to-insights/>
- [3] Working with webp and jpeg images have different number of channels. (2021). Stack Overflow. [https://stackoverflow.com/questions/67584691/working-with-webp-and-jpeg-images-have-different-number-of-channels#:~:text=Working%20with%20webp%20and%20jpeg%20images%20have%20different%20number%20of%20channels,-Ask%20Question&text=The%20problem%20is%2C%20webp%20are,in%203%20channel\(RGB\).](https://stackoverflow.com/questions/67584691/working-with-webp-and-jpeg-images-have-different-number-of-channels#:~:text=Working%20with%20webp%20and%20jpeg%20images%20have%20different%20number%20of%20channels,-Ask%20Question&text=The%20problem%20is%2C%20webp%20are,in%203%20channel(RGB).)
- [4] Tensorflow DecodeJPEG: Expected image (JPEG, PNG, or GIF), got unknown format starting with "\000\000\000\000\000\000\000\00." (2020). Stack Overflow. <https://stackoverflow.com/questions/62544528/tensorflow-decodejpeg-expected-image-jpeg-png-or-gif-got-unknown-format-st>